PM Skills Suite

一套专为产品经理设计的 AI Agent Skill 集合，覆盖从模糊想法到上线度量的完整产品工作流。

每个 Skill 都是一个**结构化工作流引擎**——不是问答机器人，而是一个有阶段、有规则、有强制产出物的协作系统，帮助 PM 在 AI 的辅助下完成真正困难的产品决策。

为什么需要这套 Skills?

产品经理的工作链路很长，但每个环节都有一个共同的敌人：

> **模糊性**——想法不清晰、数据太多但理解太少、决策没有依据、需求写完之后没人懂。

传统 AI 对话可以帮你写东西，但它无法保证你**问对了问题**、**看到了反例**、**不会 solution-first**、**每条判断都有证据**。

这套 Skills 做的事情是：**把最好的产品方法论编码进 AI 的行为规则里**，让 AI 成为一个「懂产品的协作者」，而不只是一个「执行指令的助手」。

Skills 地图

产品工作流（5 个 Skill，覆盖完整闭环）

```
模糊想法
    │
    ▼
┌──────────────────────────┐
│  Problem Discovery Engine  │   把想法 → 结构化问题模型 + 可验证假设
└──────────────────────────┘
    │
    ▼
┌──────────────────────────┐
│  User Insight Synthesizer  │   把原始用户数据 → 用户认知模型 + 机会信号
└──────────────────────────┘
    │
    ▼
┌──────────────────────────────┐
│  Feature Prioritization Engine  │   把 Backlog → 透明排序的产品路线图
└──────────────────────────────┘
    │
    ▼
┌──────────────────────────┐
│       PRD Architect        │   把决策好的功能 → 工程可执行的 PRD
└──────────────────────────┘
    │
    ▼
┌──────────────────────────┐
│   Product Metrics Analyst  │   把上线数据 → 行动建议 + 下一轮假设
```

```
└──────────────────────────────┘
        │
        ▼
（进入下一个发现周期）
```

工程理解（1 个 Skill）

```
任意代码库
    │
    ▼
┌──────────────────────────────┐
│    Project Intelligence      │  把代码库 → 系统架构 + 核心模块 + 数据流 + 复杂度
地图
└──────────────────────────────┘
    │
    ▼
Project Intelligence Report（开发者 / PM / 架构三种视角）
```

每个 Skill 可以独立触发，也可以串联使用。

Skills 一览

1. Problem Discovery Engine

解决什么问题：PM 最容易犯的错——还没搞清楚问题，就开始设计解决方案。

核心能力：

- 对话式追问，阻止「solution-first」跳跃
- 建立 User Situation Model（谁 / 什么情境 / 触发点 / 目标 / 约束）
- 提炼 JTBD（Job To Be Done）
- 五问法 / 鱼骨图根因分析
- 把机会空间转为可证伪的产品假设

产出物：

- `Problem Discovery Brief`（结构化问题模型）
- `Hypothesis Ledger`（假设台账，含置信度与下一步验证计划）

---

2. User Insight Synthesizer

解决什么问题：做了 20 个用户访谈，结果每个人说的都不一样，团队陷入"数据有了但理解没有"的迷雾。

核心能力：

- 把原始文本拆成 atomic signals 并编码
- 主题聚类（Thematic Analysis）发现行为模式
- 识别 User Archetypes（行为人格）

- Motivation Ladder：从表面需求挖到核心动机
- 强制标注少数声音，防止确认偏误

产出物：

- `User Insight Report`（含 User Segments / Behavior Patterns / Pain Points / Hidden Needs / Motivations / Opportunity Signals，每条 Insight 附 Evidence）

---

3. Feature Prioritization Engine

解决什么问题：优先级讨论变成政治博弈——谁声音大谁赢，没有统一标准，没有显性的 trade-off。

核心能力：

- 统一评估标准（User Value / Business Impact / Strategic Alignment / Effort / Risk）
- 支持 RICE / ICE / MoSCoW / Impact-Effort 等框架
- 透明评分（每条分数都有明确理由）
- Quick Win vs Strategic Bet vs Defer 分类
- 组合视角：建立平衡路线图，而不只是按分数排列

产出物：

- `Prioritized Feature Roadmap`（含评分依据、trade-off 说明、风险项）
- `Feature Scoring Table`（CSV 格式，可导入表格工具）

---

4. PRD Architect

解决什么问题：PRD 写完之后，工程师说"看不懂"，设计师说"没讲清楚边界"，测试说"没有验收标准"。

核心能力：

- User Story 构建（As a / I want to / So that）
- 功能需求写成可测试的 shall/should 语句
- 边界条件与异常流程矩阵（Edge Case Matrix）
- 非功能需求（性能 / 安全 / 可靠性）量化阈值
- 成功指标定义（指标 + 目标值 + 数据来源）
- 完整性检查清单（输出前强制过一遍）

产出物：

- `Product Requirements Document`（Engineer-ready PRD）
- `Edge Case Matrix`
- `Metrics Table`

---

5. Product Metrics Analyst

解决什么问题：功能上线了，PM 盯着数据大盘不知道该看什么、为什么会这样、下一步该做什么。

核心能力：

- 确立分析框架（北极星指标 + 分层指标）
- 趋势检测 + 异常归因（内置 Anomaly Heuristics）
- 漏斗分析 + 激活路径 + Drop-off 识别
- 把 observation 转化为 testable hypothesis
- 推荐最小验证实验（含成功标准）

产出物：

- `Product Performance Insight Report`
- `Hypothesis Log`（CSV）
- `Metrics Snapshot Table`（CSV）

---

6. Project Intelligence

解决什么问题：接手一个新代码库，README 写得很漂亮，但你依然不知道系统怎么运作、数据从哪来、哪个模块是核心、改一行代码会不会炸。

核心能力：

- 自动识别核心模块，并赋予"系统角色"（Orchestrator / Gatekeeper 等）
- 追踪关键请求流：从用户操作到数据库的完整路径
- 把代码逻辑翻译成业务叙事（Business Scenarios）
- 解读数据模型的产品意义
- 生成复杂度热力图，标注高危文件
- 输出新人学习路径（Onboarding Path）
- 支持三种角色视角：Developer / PM / Architecture

产出物：

- `Project Intelligence Report`（含 9 个部分，覆盖架构 / 模块 / 数据流 / 业务场景 / 复杂度 / Onboarding）

如何使用

这套 Skills 需要安装到 AI 工具中才能生效。目前支持两个工具：**Cursor**（AI 代码编辑器）和 **Codex**（OpenAI 命令行 Agent）。按照你正在使用的工具选择对应流程。

> **系统要求**：macOS 或 Linux。Windows 用户请参考文末说明。

第一步：下载这个仓库

打开终端，把仓库克隆到本地：

```
git clone https://github.com/<仓库地址> ~/Documents/Skills
```

> 如果你不熟悉 Git，也可以点击 GitHub 页面右上角的 **Code → Download ZIP**，解压后放到你喜欢的位置（记住这个路径，后续步骤会用到）。

克隆完成后，进入目录确认文件存在：

```
ls ~/Documents/Skills
```

你应该能看到 `Problem Discovery Engine`、`Feature Prioritization Engine` 等文件夹。

第二步：选择你的 AI 工具

方案 A：在 Cursor 中使用（推荐）

Cursor 是一款内置 AI Agent 的代码编辑器。Skills 安装后，在任意项目的对话中均可自动触发。

2A-1：确认已安装 Cursor

前往 cursor.com 下载并安装。打开后确认可以正常启动。

2A-2：创建个人 Skills 目录

Cursor 会从 `~/.cursor/skills/` 目录中自动加载 Skills。先创建这个目录：

```
mkdir -p ~/.cursor/skills
```

> `~` 是你的用户主目录的简写（macOS 上通常是 `/Users/你的用户名`）。`-p` 表示如果目录已存在也不报错。

2A-3：安装 Skills（创建符号链接）

符号链接（symlink）相当于一个"快捷方式"——它让 Cursor 能读取到仓库里的文件，同时你在仓库中做的任何修改会立即生效，无需重新安装。

> ⚠ **注意**：如果你把仓库克隆到了其他位置，请把下面命令中的 `~/Documents/Skills` 替换成你实际的路径。

```
# 安装全部 6 个 Skills
ln -s ~/Documents/Skills/Problem\ Discovery\ Engine/problem-discovery-engine         ~/.cursor/skills/problem-discovery-engine
ln -s ~/Documents/Skills/User\ Insight\ Synthesizer/user-insight-synthesizer         ~/.cursor/skills/user-insight-synthesizer
ln -s ~/Documents/Skills/Feature\ Prioritization\ Engine/feature-prioritization-engine ~/.cursor/skills/feature-prioritization-engine
ln -s ~/Documents/Skills/PRD\ Architect/prd-architect ~/.cursor/skills/prd-architect
ln -s ~/Documents/Skills/Product\ Metrics\ Analyst/product-metrics-analyst ~/.cursor/skills/product-metrics-analyst
```

```
ln -s ~/Documents/Skills/项目理解引擎/project-intelligence
~/.cursor/skills/project-intelligence
```

也可以只安装你需要的 Skill，每行命令独立可用。

2A-4：验证安装

运行以下命令，确认符号链接都已正确创建：

```
ls -la ~/.cursor/skills/
```

正常输出类似：

```
lrwxr-xr-x  problem-discovery-engine -> /Users/你的用户
名/Documents/Skills/...
lrwxr-xr-x  feature-prioritization-engine -> ...
...
```

每一行开头的 `l` 表示这是一个符号链接，说明安装成功。

2A-5：在 Cursor 中触发 Skill

无需重启。打开 Cursor，在任意项目中打开 AI 对话（`Cmd+L` 或右侧 Chat 面板），直接描述你的任务即可：

```
我有一个需求想法，但还不确定问题是什么，帮我用 problem-discovery-engine 分析一下
```

```
这是我们 10 个用户访谈的原始记录，用 user-insight-synthesizer 帮我提炼用户洞察
```

```
分析这个代码库，用 project-intelligence 生成一份 Project Intelligence Report
```

Cursor 会自动读取对应 Skill 的工作流规则，按结构化流程引导你完成任务。

> ⚠ 不要把文件放入 `~/.cursor/skills-cursor/`，那是 Cursor 的内置系统目录，手动修改会导致异常。

方案 B：在 Codex 中使用

Codex 是 OpenAI 推出的命令行 AI Agent 工具。

2B-1：确认已安装 Codex

```
codex --version
```

如果提示"命令未找到"，请先参考 Codex 官方文档 完成安装。

2B-2：安装 Skills

```
mkdir -p ~/.codex/skills

ln -s ~/Documents/Skills/Problem\ Discovery\ Engine/problem-discovery-
engine         ~/.codex/skills/problem-discovery-engine
ln -s ~/Documents/Skills/User\ Insight\ Synthesizer/user-insight-
synthesizer         ~/.codex/skills/user-insight-synthesizer
ln -s ~/Documents/Skills/Feature\ Prioritization\ Engine/feature-
prioritization-engine ~/.codex/skills/feature-prioritization-engine
ln -s ~/Documents/Skills/PRD\ Architect/prd-architect
~/.codex/skills/prd-architect
ln -s ~/Documents/Skills/Product\ Metrics\ Analyst/product-metrics-analyst
~/.codex/skills/product-metrics-analyst
ln -s ~/Documents/Skills/项目理解引擎/project-intelligence
~/.codex/skills/project-intelligence
```

2B-3：重启 Codex

与 Cursor 不同，Codex 需要**重启**后才会加载新安装的 Skills。

2B-4：触发 Skill

启动 Codex 后直接描述任务，它会自动匹配对应 Skill。也可以明确指定：

```
用 feature-prioritization-engine 帮我对这 6 个功能排优先级
```

Windows 用户说明

Windows 不支持上述 `ln -s` 符号链接命令。有两种替代方案：

方案一（推荐）：使用 WSL

在 WSL（Windows Subsystem for Linux）中操作，命令与上述完全一致。

方案二：直接复制文件夹

```
# 在 PowerShell 中运行（以 Cursor 为例）
mkdir $env:USERPROFILE\.cursor\skills
Copy-Item "C:\你的路径\Skills\Feature Prioritization Engine\feature-
prioritization-engine" `
```

```
  -Destination "$env:USERPROFILE\.cursor\skills\feature-prioritization-engine" -Recurse
```

注意：直接复制的文件不会随仓库更新自动同步，需要手动重新复制。

串联使用示例

场景：从一个模糊想法出发，一路做到上线度量

```
1. 「我们要做一个 AI 会议摘要功能」
   → Problem Discovery Engine
   → 产出：Problem Brief + Hypothesis Ledger

2. 「这是我们收集到的 30 个用户访谈记录」
   → User Insight Synthesizer
   → 产出：User Insight Report（包含 User Archetypes + Opportunity Signals）

3. 「结合 Brief 和 Insight Report，现在有 8 个功能候选」
   → Feature Prioritization Engine
   → 产出：Prioritized Roadmap + Scoring Table

4. 「会议摘要功能排名第一，现在需要写需求」
   → PRD Architect
   → 产出：Engineer-ready PRD + Edge Case Matrix

5. 「功能上线两周，这是数据」
   → Product Metrics Analyst
   → 产出：Performance Insight Report + 下一轮假设与实验建议
```

设计原则

这套 Skills 遵循四个核心设计原则：

1. Evidence-first（证据优先） 每条判断、每个 insight、每项需求都必须附带证据来源。推断必须显式标注为推断。

2. Forced structure（强制结构） 每个 Skill 都有明确的 Stage 和 Exit Criteria。AI 不能跳跃阶段，用户也不能在问题没搞清楚之前就进入方案讨论。

3. Minority tracking（少数声音保留） 主流观点旁必须记录反例和少数声音，防止确认偏误。这是大多数 PM 工具缺失的能力。

4. Explicit uncertainty（显式不确定性） 用 Low / Medium / High 置信度标注每条结论，并明确「数据缺口」和「下一步需要验证什么」。

目录结构

```
Skills/
├── Problem Discovery Engine/         # Skill 1: 发现 & 定义问题
├── User Insight Synthesizer/         # Skill 2: 用户洞察分析
├── Feature Prioritization Engine/    # Skill 3: 功能排优先级
│   └── feature-prioritization-engine/
├── PRD Architect/                    # Skill 4: 撰写需求文档
│   └── prd-architect/
├── Product Metrics Analyst/          # Skill 5: 上线数据分析
│   └── product-metrics-analyst/
└── 项目理解引擎/                      # Skill 6: 代码库深度理解
    └── project-intelligence/
```

每个 Skill 目录结构：

```
skill-name/
├── SKILL.md                    # 核心：工作流 + 操作规则
├── agents/openai.yaml          # UI 元数据（display name, default prompt）
├── references/                 # 方法论参考 + 分阶段题库
├── assets/                     # 输出物模板（Report / PRD / Roadmap 等）
└── scripts/
    └── validate_skill.py       # 零依赖结构校验脚本
```

贡献 & 迭代

每个 Skill 都设计为**可独立迭代**的模块。如需改进某个 Skill：

1. 修改对应 `SKILL.md` 或 `references/` 文件
2. 运行校验脚本确认格式正确：

   ```
   python3 "path/to/skill/scripts/validate_skill.py" "path/to/skill"
   ```

3. 在真实任务中测试，根据使用体验调整

> 这套 Skills 本身就是真人以及 skill-creator 协作构建的。